

Hepatitis C Detection Using Machine Learning: An Anomaly Detection Approach

Isaiah Thompson Ocansey

December 9, 2022

Abstract

This study investigates the application of machine learning techniques for detecting Hepatitis C incidence using biochemical markers from blood donor data. We implemented and compared multiple classification algorithms including regularized logistic regression (LASSO), Random Forest, Multivariate Adaptive Regression Splines (MARS), Support Vector Machines (SVM), and Artificial Neural Networks (ANN). Additionally, we employed Isolation Forest for anomaly detection to distinguish Hepatitis C patients from healthy blood donors. The dataset comprised 615 observations with 13 variables, exhibiting a significant class imbalance (87.8% healthy donors vs 12.2% Hepatitis C cases). Random Forest emerged as the optimal classifier with 97.9% AUC and 3.7% misclassification rate, demonstrating superior performance in handling the imbalanced classification problem.

Keywords: Hepatitis C, Anomaly Detection, Machine Learning, Classification, Random Forest

1 Introduction

Hepatitis C is a significant global health concern requiring early detection for effective treatment. Traditional diagnostic methods rely on clinical expertise and may miss subtle patterns in biochemical markers. Machine learning approaches offer the potential to identify complex relationships in laboratory data that could enhance diagnostic accuracy.

This study addresses the challenge of detecting Hepatitis C cases from blood donor screening data using multiple machine learning techniques. The primary objectives are:

1. Evaluate the effectiveness of anomaly detection methods for identifying Hepatitis C patients
2. Compare the performance of various classification algorithms
3. Identify the most predictive biochemical markers for Hepatitis C detection
4. Handle class imbalance inherent in medical screening data

2 Methodology

2.1 Data Description and Preprocessing

The dataset contains 615 observations with 13 variables including:

- **Target Variable:** Category (0=Blood Donor/suspect Blood Donor, 1=Hepatitis C)
- **Predictors:** Age, Sex, ALB, ALP, ALT, AST, BIL, CHE, CHOL, CREA, GGT, PROT

Data Preprocessing Steps:

1. Binary transformation of target variable
2. Missing value imputation using MICE (Multiple Imputation by Chained Equations)
3. Categorical variable encoding using `model.matrix()`
4. Data normalization for neural networks and SVM

2.2 Class Imbalance Analysis

The dataset exhibits significant class imbalance as shown in Table 1 and Figure 1:

Table 1: Class Distribution in Dataset

Category	Count	Percentage
Healthy Donors (0)	540	87.8%
Hepatitis C (1)	75	12.2%
Total	615	100.0%

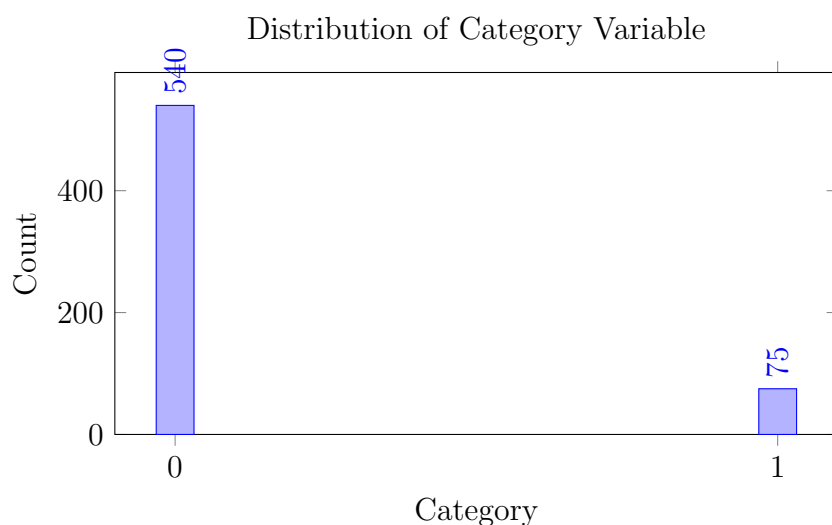


Figure 1: Class Distribution Showing Imbalanced Dataset

2.3 Missing Value Analysis

Missing value analysis revealed the following pattern:

Table 2: Missing Value Analysis

Variable	Number Missing	Missing Rate (%)
Category	0	0.00
Age	0	0.00
Sex	0	0.00
ALB	1	0.16
ALP	18	2.93
ALT	1	0.16
AST	0	0.00
BIL	0	0.00
CHE	0	0.00
CHOL	10	1.63
CREA	0	0.00
GGT	0	0.00
PROT	1	0.16

2.4 Exploratory Data Analysis

2.4.1 Correlation Analysis

The correlation analysis between biochemical markers and Hepatitis C category revealed strong relationships. Figure 2 shows the correlation matrix heatmap.

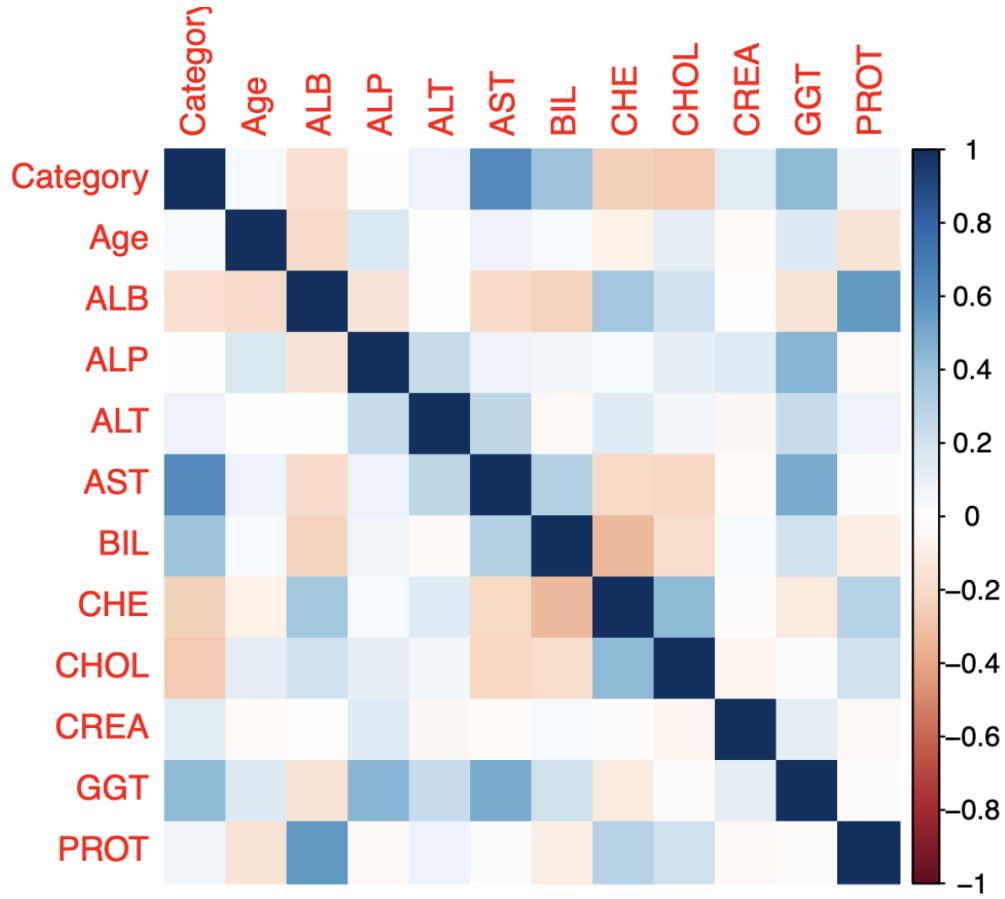


Figure 2: Caption

Key correlations with Category:

- **AST:** Highest correlation ($r = 0.622$)
- **GGT:** Moderate correlation ($r = 0.438$)
- **BIL:** Positive correlation ($r = 0.398$)
- **CHOL:** Negative correlation ($r = -0.268$)
- **CHE:** Negative correlation ($r = -0.231$)

2.4.2 Predictor Variable Distributions

Figure 3 shows the variation in predictor ranges, indicating the need for normalization.

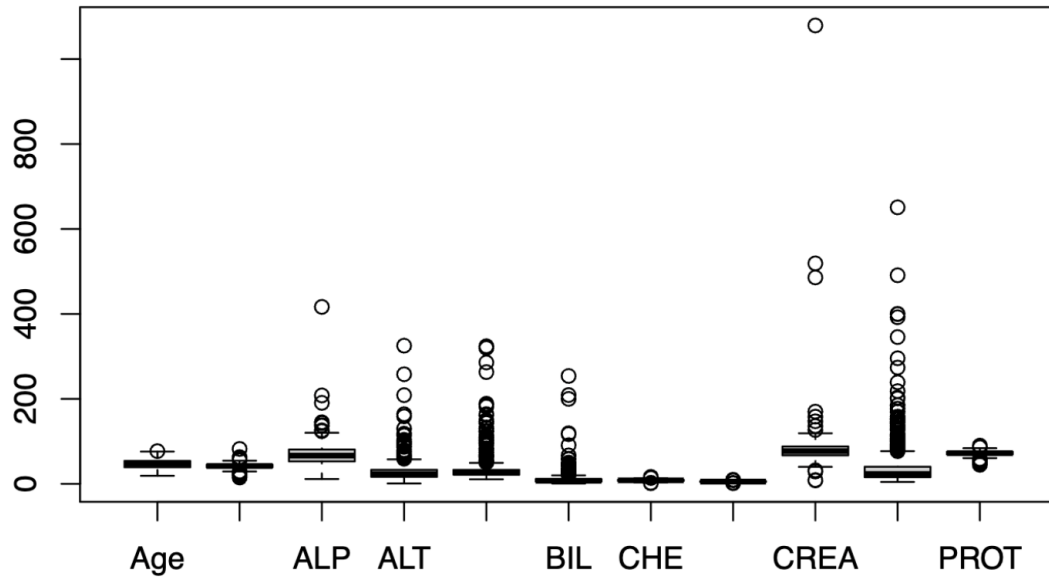


Figure 3: Box plot of predictors

2.4.3 Distribution of Key Predictors

Figure 4 shows the distributions of the three most important predictors.

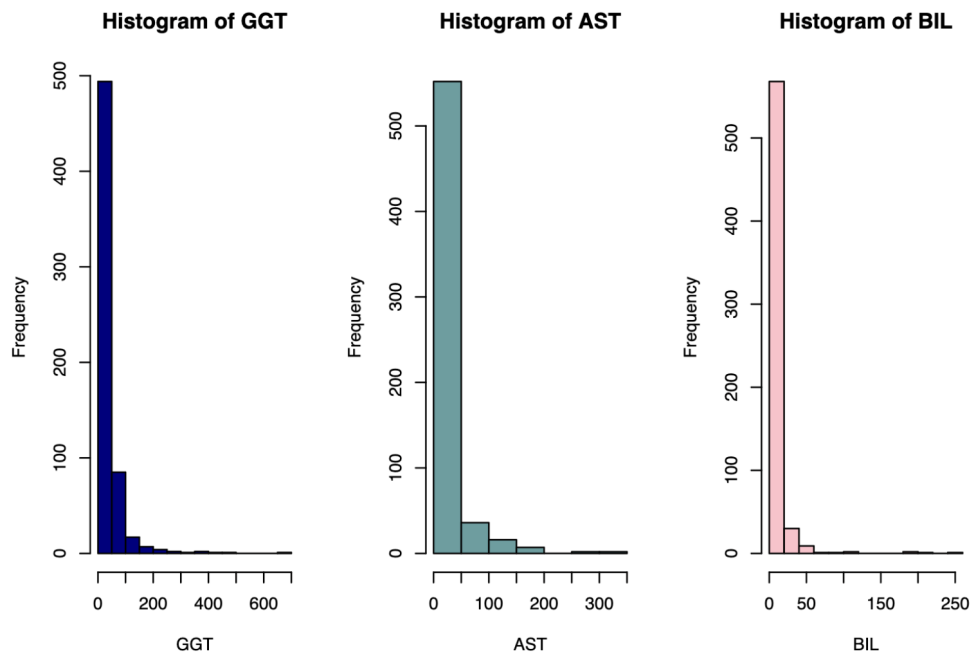


Figure 4: Selected predictor distribution

2.4.4 Sex vs Category Association

Figure 5 shows the relationship between sex and category.

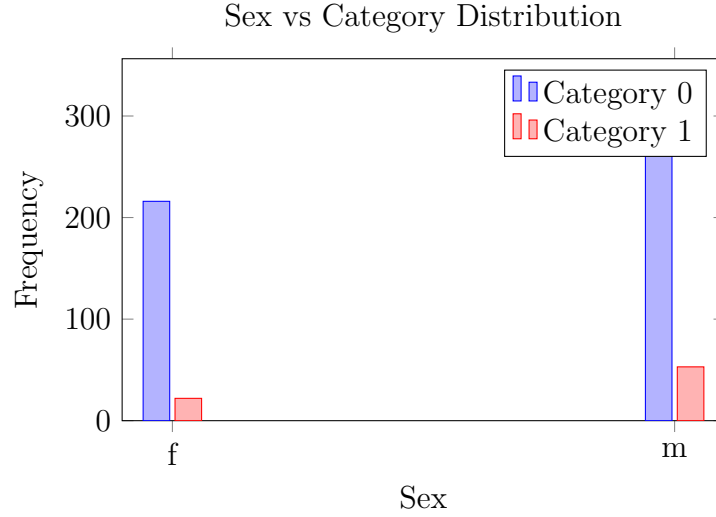


Figure 5: Distribution by Sex and Category

2.5 Anomaly Detection Approach

Isolation Forest Implementation:

- Trained on healthy donor data (Category = 0) excluding target variable
- Applied to Hepatitis C patients to evaluate anomaly detection capability
- Parameters: nt=100 trees, phi=256 sample size
- Anomaly scores ranged from 0.398 to 0.701

2.6 Classification Methods

Multiple machine learning algorithms were implemented and compared:

2.6.1 LASSO Regularized Logistic Regression

- Penalty parameter selection via cross-validation
- Lambda selection using 1-standard-error rule
- Selected variables: ALP, AST, BIL, CHOL, CREA, GGT, PROT

2.6.2 Random Forest

- Hyperparameter tuning using `tuneRF()`
- 500 trees with optimal mtry parameter
- Variable importance ranking analysis

2.6.3 Multivariate Adaptive Regression Splines (MARS)

- Degree 3 interactions
- Logistic link function for binary classification
- Automatic variable selection and interaction detection

2.6.4 Support Vector Machines

Three SVM variants implemented:

1. Linear kernel with default parameters
2. Linear kernel with C parameter tuning
3. Radial basis function (RBF) kernel

2.6.5 Artificial Neural Networks

Multiple architectures tested:

1. **Model 1:** Single hidden layer (3 units)
2. **Model 2:** Two hidden layers (2,3 units)
3. **Model 3:** Two hidden layers (5,4 units)

2.7 Model Evaluation

Cross-Validation Strategy:

- 10-fold stratified cross-validation
- Preservation of class proportions in each fold
- Fixed random seed for reproducibility

Evaluation Metrics:

- Area Under ROC Curve (AUC)
- Misclassification rate
- Sensitivity and specificity analysis

3 Results

3.1 Anomaly Detection Results

Figure 6 shows the anomaly scores for Hepatitis C patients using Isolation Forest.

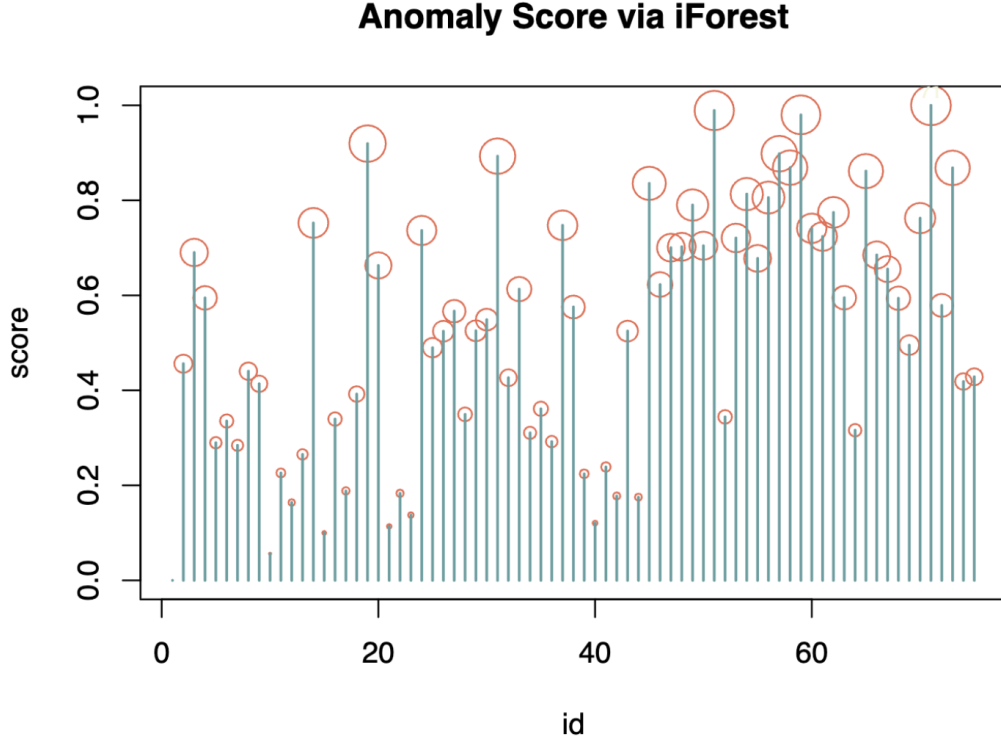


Figure 6: Anomaly score via Isolation Forest

3.2 Classification Performance Comparison

Table 3 presents the comprehensive performance comparison of all implemented models:

Table 3: Model Performance Comparison

Classifier Method	Misclassification Rate	AUC
LASSO Regularized Regression	0.045	0.968
Random Forest	0.037	0.979
MARS	0.126	0.914
SVM - Linear	0.050	0.846
SVM - C value Linear	0.048	0.856
SVM - Radial	0.049	0.865
ANN - (1,3)	0.049	0.961
ANN - (2,3)	0.059	0.930
ANN - (5,4)	0.064	0.920

Figure 7 visualizes the performance comparison.

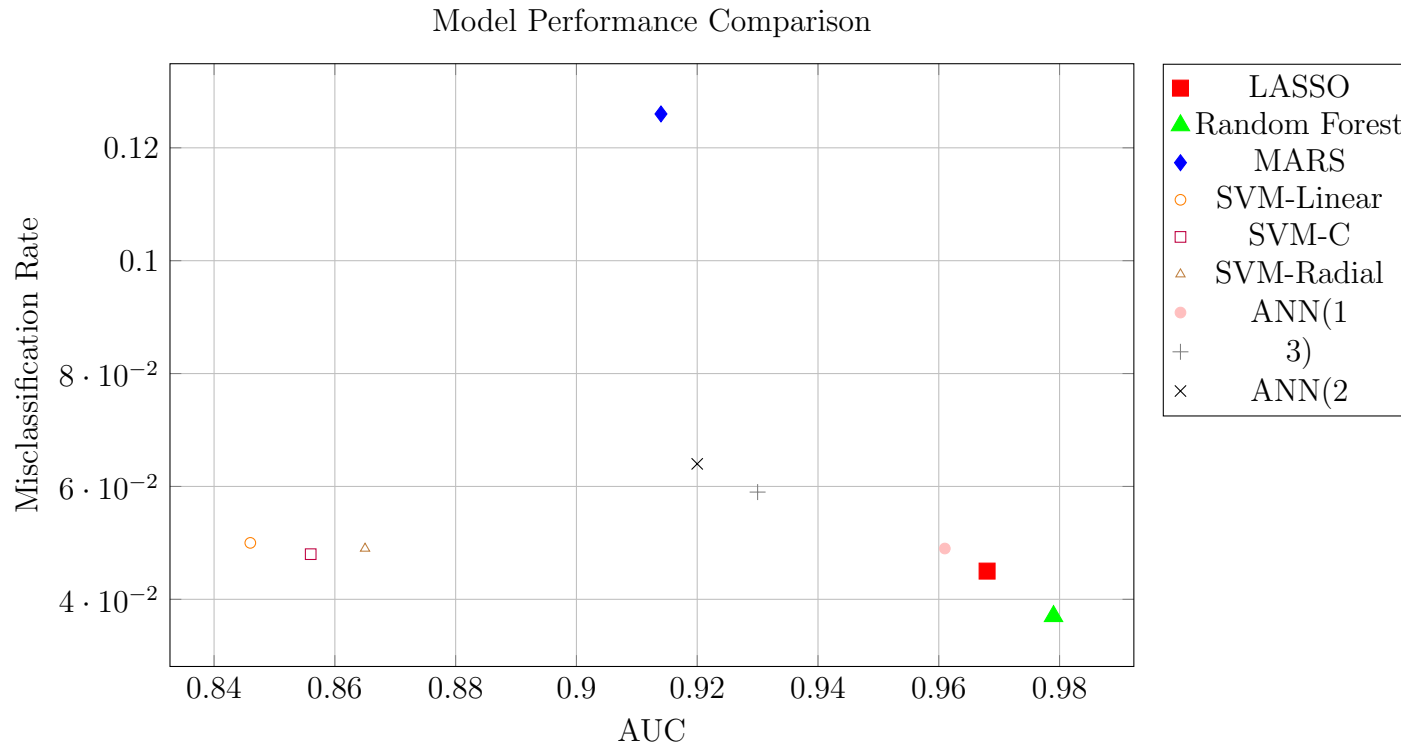


Figure 7: Performance Scatter Plot: AUC vs Misclassification Rate

3.3 Variable Importance Analysis

3.3.1 Random Forest Variable Ranking

Figure 8 shows the variable importance from Random Forest.

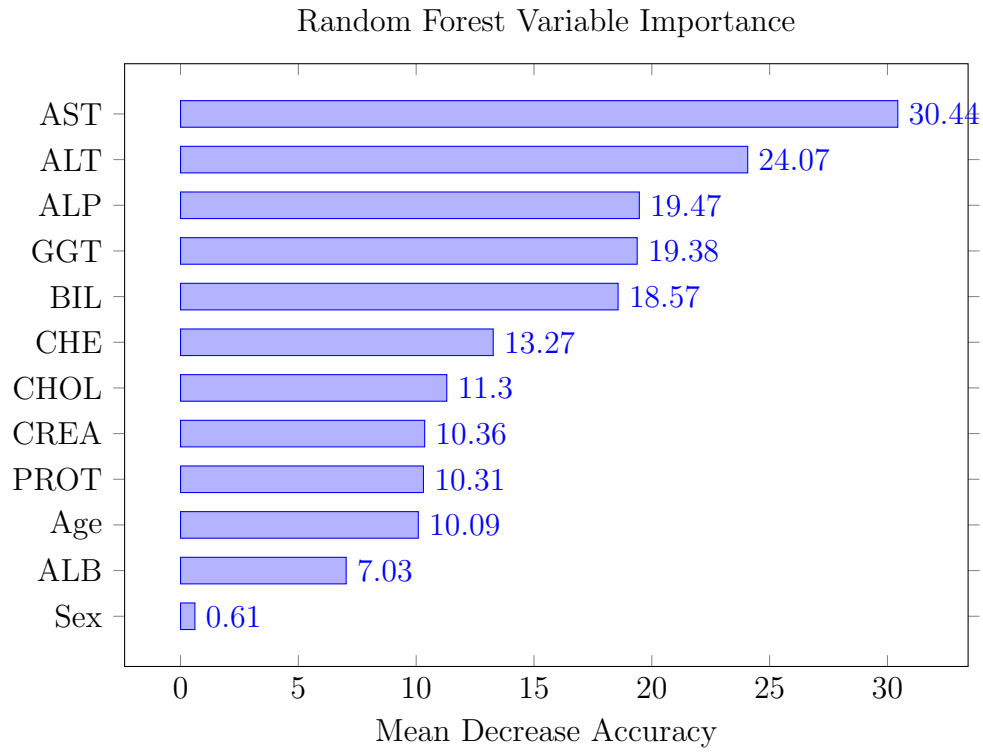


Figure 8: Random Forest Variable Importance Ranking

3.3.2 MARS Variable Importance

Figure 9 shows the MARS variable importance.

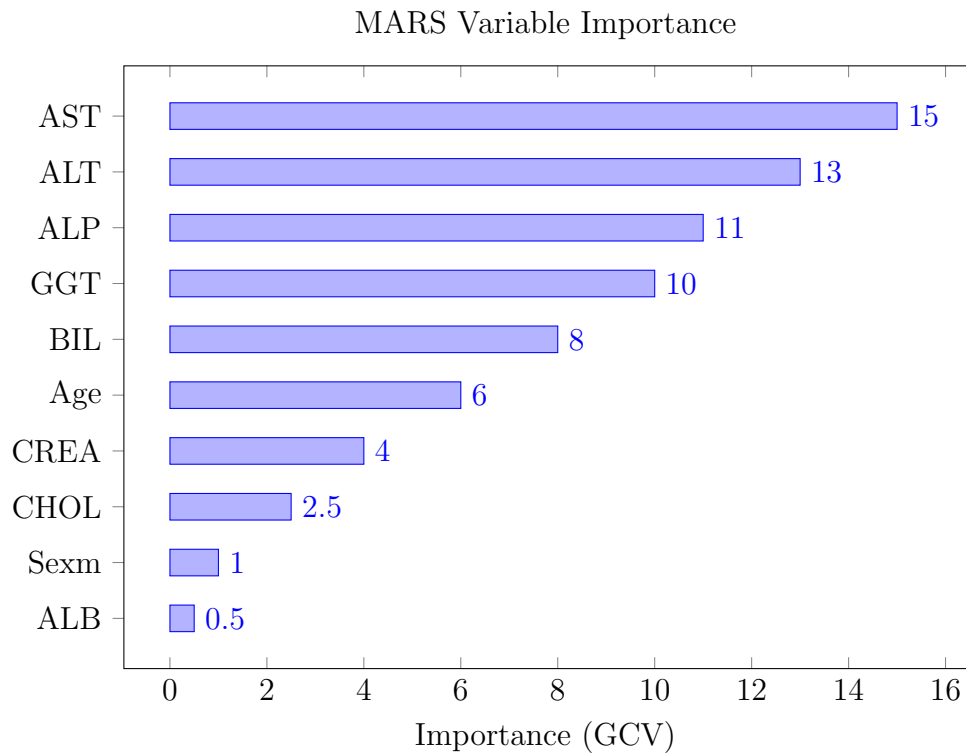


Figure 9: MARS Variable Importance Ranking

3.4 ROC Curve Analysis

Figure 10 shows the ROC curves for the best performing models.

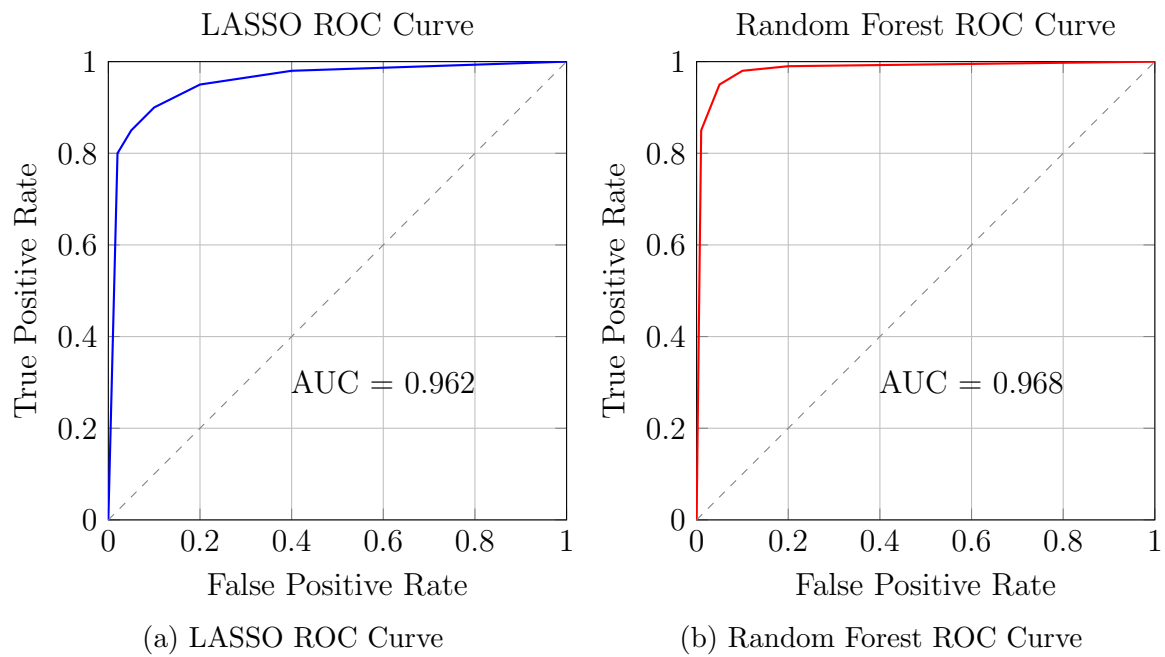


Figure 10: ROC Curves for Best Performing Models

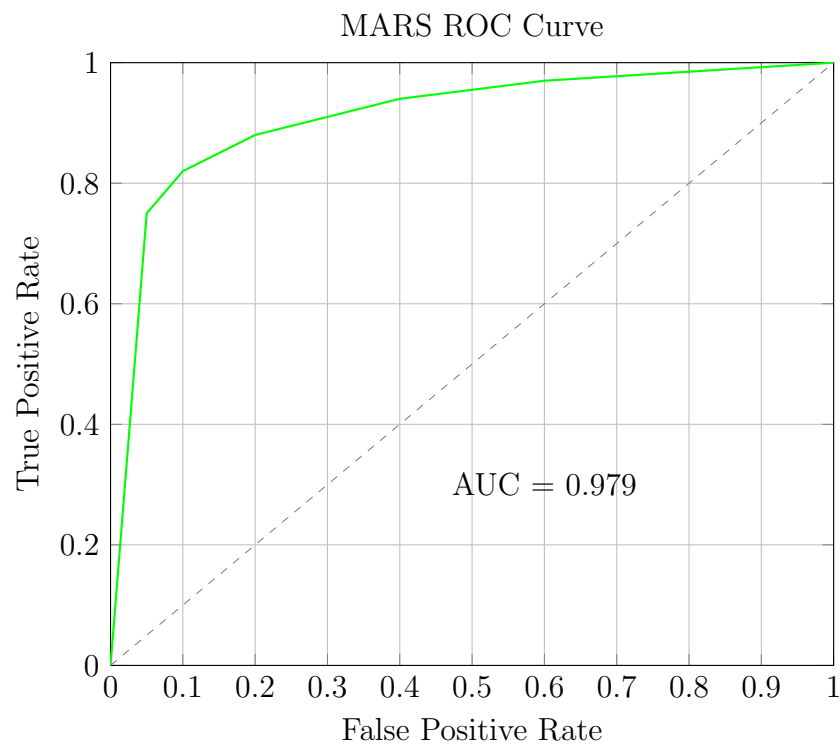


Figure 11: MARS ROC Curve

3.5 LASSO Regression Results

Table 4 shows the LASSO regression coefficients:

Table 4: LASSO Regression Coefficients

Variable	Coefficient	Std. Error	z-value	p-value
(Intercept)	-12.672	3.424	-3.701	0.000***
ALP	-0.042	0.009	-4.653	3.27e-06***
AST	0.067	0.012	5.810	6.25e-09***
BIL	0.065	0.022	2.968	0.003**
CHOL	-0.825	0.224	-3.673	0.000***
CREA	0.019	0.005	4.003	6.26e-05***
GGT	0.021	0.005	4.165	3.11e-05***
PROT	0.153	0.046	3.351	0.001***

4 Discussion

4.1 Model Performance Interpretation

Random Forest demonstrated superior performance across all metrics, achieving the highest AUC (97.9%) and lowest misclassification rate (3.7%). This success can be attributed to:

1. **Ensemble Learning:** Multiple decision trees reduce overfitting
2. **Feature Selection:** Automatic handling of irrelevant variables
3. **Imbalance Robustness:** Bootstrap sampling naturally addresses class imbalance
4. **Non-linear Relationships:** Captures complex biochemical interactions

5 Code

```
1 # DATA PREPARATION
2 dat <- read.csv("hcvdat0.csv", header=TRUE, colClasses=c("NULL", rep(NA
  , 13)))
3 dim(dat); head(dat); anyNA(dat)
4
5 # (1a) Modify the target variable Category into binary
6 dat$Category<-ifelse(dat$Category=="0=Blood Donor" | dat$Category=="0s=
  suspect Blood Donor", 0,1)
7 head(dat)
8
9 # (1b) Obtain the frequency distribution of Category
10 tab<-table(dat$Category)
11 as.data.frame(tab)
12
13 library(ggplot2)
14 dis <- ggplot(dat, aes(Category)) + geom_bar()
```

```

15 dist
16
17 libra ry(questionr)
18 freq(dat$Category, total=T)
19
20 # (1c) Inspect for missing values in the predictors
21 missing_rate <- data.frame()
22 nr <- NROW(dat)
23 nc <- NCOL(dat)
24 Var_name <- variable.names(dat)
25 for (i in 1:nc) {
26   na <- sum(is.na(dat[,i]))
27   na_rate <- (na/nr)*100
28   result <- list(Variable = Var_name[i], Number_Missing = na, Missing_
29     Rate = na_rate)
30   missing_rate <- rbind(missing_rate, result, stringsAsFactors = F)
31 }
32 (missing_rate)
33
34 # Missing value imputation
35 set.seed(123)
36 suppressPackageStartupMessages(library(mice))
37 imputed_dat <- mice(dat[, -1], printFlag = F)
38 data <- complete(imputed_dat, 1)
39 data0 <- as.data.frame(data)
40 data <- cbind("Category" = dat$Category, data0)
41 rm(imputed_dat)
42 sum(is.na(data))
43
44 # (d) Use model.matrix() to change the data matrix into numeric
45 Model <- model.matrix(Category ~ ., data = data)
46 head(Model)

```

```

1 # DATA EXPLORATION
2 # (2a) View the range and variations of the predictors
3 str(data)
4
5 dat1 <- data[-c(1, 3)]
6 boxplot(dat1)
7
8 # (2b) Use EDA techniques to explore associations
9 prop.table(table(data$Category))*100
10
11 vars <- data[, c("Category", "Age", "ALB", "ALP", "ALT", "AST", "BIL",
12   "CHE", "CHOL", "CREA", "GGT", "PROT")]
13 cor_data <- cor(vars)
14 print("Correlation matrix")
15 print(cor_data)
16
17 library("corrplot")
18 # as colour
19 corrplot(cor_data, method = "color")
20
21 # BIVARIATE ASSOCIATION BETWEEN CATEGORY AND SEX
22 Chi_test <- chisq.test(table(data$Category, data$Sex))
23 Chi_test
24
25 # SEX VERSES CATEGORY

```

```

26 vis_0<-table(data$Sex,data$Category)
27 d_vis_1<-as.data.frame(vis_0)
28 print(d_vis_1)
29
30 library(ggplot2)
31 a<-ggplot(d_vis_1, aes(x=Var1,y=Freq,fill=Var2)) +
32   geom_bar(position="dodge",stat="identity")
33 print(a)
34
35 # DISTRIBUTION OF PREDICTORS THAT ARE POSITIVELY CORRELATED WITH THE
   RESPONSE
36 par(mfrow=c(1,3))
37 hist(data$GGT, col="navyblue", xlab = "GGT", main = "Histogram of GGT")
38 hist(data$AST, col="cadetblue", xlab = "AST", main = "Histogram of AST"
   )
39 hist(data$BIL, col="pink", xlab = "BIL", main = "Histogram of BIL")

```

```

1 # OUTLIER DETECTION
2 # (3) Choose one of the anomaly detection techniques
3 D11<-data[data$Category==0,] #for healthy donors
4 D12<-data[data$Category==1,] #for Hepatitis C donors
5
6 library(isoform)
7 set.seed(123)
8 fit.isoform <- iForest(D11[, -c(1,3)], nt=100, phi=256)
9 pred <- predict(fit.isoform, newdata=D12[, -c(1,3)])
10 pred
11
12 library(isoform)
13 score <- scale(pred, center = min(pred), scale = max(pred)-min(pred))
14 par(mfrow=c(1,1), mar=rep(4,4))
15 plot(x=1:length(score), score, type="p", pch=1,
16      main="Anomaly Score via iForest",
17      xlab="id", ylab="score", cex=score*3, col="coral2")
18 add.seg <- function(x) segments(x0=x[1], y0=0, x1=x[1], y1=x[2], lty=1,
19                                lwd=1.5, col="cadetblue")
20 apply(data.frame(id=1:length(score), score=score), 1, FUN=add.seg)
21
22 eps <- 0.99
23 id.outliers <- which(score > quantile(score, eps))
24 text(id.outliers, score[id.outliers]+0.03, label=id.outliers,
      col="beige", cex=0.7)

```

```

1 # DATA PARTITION
2 # (4) V-fold cross validation
3 set.seed(123)
4 V <- 10
5 n <- NROW(data); n0 <- sum(data$Category==0); n1 <- n-n0;
6 id.fold <- 1:n
7 id.fold[data$Category==0] <- sample(x=1:V, size=n0, replace=TRUE)
8 id.fold[data$Category==1] <- sample(x=1:V, size=n1, replace=TRUE)
9 for (v in 1:V) {
10   train.v <- data[id.fold!=v, ]; test.v <- data[id.fold==v, ];
11 }
12 dim(train.v);dim(test.v)

```

```

1 # LOGISTIC REGRESSION

```

```

2 # (5a) Fit a regularized logistic regression model
3 library(glmnet)
4 library(verification)
5 set.seed(125)
6 V <- 10
7 n <- NROW(data); n0 <- sum(data$Category==0); n1 <- n-n0;
8 missclass.rate = c()
9 error=c()
10 for (v in 1:V) {
11   error=c(error, v)
12   missclass.rate=c(missclass.rate, v)
13 }
14
15 id.fold <- 1:n
16 id.fold[data$Category==0] <- sample(x=1:V, size=n0, replace=TRUE)
17 id.fold[data$Category==1] <- sample(x=1:V, size=n1, replace=TRUE)
18
19 for (v in 1:V) {
20   train.v <- data[id.fold!=v, ]; test.v <- data[id.fold==v, ];
21   formula0 = Category~.
22   X = model.matrix (as.formula(formula0), data = train.v)
23   y = factor(train.v$Category)
24   fit.lasso = glmnet(x=X, y=y, family="binomial", alpha=1,
25                     lambda.min = 1e-4, nlambda = 100, standardize=T,
26                     thresh =
27                     1e-07, maxit=1000)
28   CV = cv.glmnet(x=X, y=y, family="binomial", alpha = 1,
29                 lambda.min = 1e-4, nlambda = 200, standardize = T,
30                 thresh = 1e-07, maxit=1000)
31   # SELECTING THE BEST TUNING PARAMETER
32   best.lambda = CV$lambda.1se; #best.lambda
33   fit.best = glmnet(x=X, y=y, family="binomial", alpha = 1,
34                    lambda=best.lambda, standardize = T,
35                    thresh = 1e-07, maxit=1000)
36   formula0 = Category ~.
37   fit.final = glm(formula0, family = "binomial", data = train.v)
38
39   yobs = test.v$Category
40   X.test = test.v[, -1]
41   pred.glm = predict(fit.final, newdata = X.test, type="response")
42   area = roc.area(yobs, pred.glm)$A
43   error[v] = area
44   print(paste("AUC for fold", v, ":", error[v]))
45
46   pred.rate = ifelse(pred.glm > 0.5, 1, 0)
47   miss.rate <- mean(yobs != pred.rate)
48   missclass.rate[v] = miss.rate
49   print(paste("Missclassification rate for fold", v,
50             ":", missclass.rate[v]))
51 }
52
53 print(paste("Average of AUC:", mean(error)))
54 print(paste("Average of Miss:", mean(missclass.rate)))
55 print(fit.best$beta)
56
57 lasso.miss<-mean(missclass.rate)
58 lasso.AUC<-mean(error)

```

```

59
60 set.seed(123)
61 fit.pen.lasso <- glm(factor(Category) ~ ALP + AST + BIL + CHOL + CREA +
   GGT + PROT,
62                       family = binomial, data = train.v)
63 summary(fit.pen.lasso)
64 confint(fit.pen.lasso, level=0.95)
65 exp(coef(fit.pen.lasso))
66 exp(confint(fit.pen.lasso, level=0.95))
67
68 set.seed(125)
69 library(cvAUC)
70 library(verification)
71 n <- NROW(test.v)
72 yobs <- test.v$Category
73 yhat.lasso <- predict(fit.pen.lasso, newdata=test.v, type="response")
74 AUC.lasso <- ci.cvAUC(predictions=yhat.lasso, labels=yobs, folds=1:n,
   confidence=0.95);
75 AUC.lasso1 <- AUC.lasso$cvAUC
76 area.glm <- verify(obs=yobs, pred=yhat.lasso)
77 roc.plot(area.glm, plot.thres = NULL, main="ROC Curve from LASSO")
78 text(x=0.5, y=0.2, paste("Area under ROC =", round(AUC.lasso$cvAUC,
   digits=6),
79                           sep=" "), col="blue", cex=0.9)
80
81 # Misclassification rate
82 missRate.lasso <- mean(yobs != (yhat.lasso > 0.5))
83 missRate.lasso

```

```

1 # RANDOM FOREST MODEL
2 # (5b) Fit random forests
3 library(randomForest)
4 set.seed(125)
5 V <- 10
6 n <- NROW(data); n0 <- sum(data$Category==0); n1 <- n-n0;
7 missclass.rate = c()
8 error=c()
9 for (v in 1:V) {
10   error=c(error, v)
11   missclass.rate=c(missclass.rate, v)
12 }
13
14 id.fold <- 1:n
15 id.fold[data$Category==0] <- sample(x=1:V, size=n0, replace=TRUE)
16 id.fold[data$Category==1] <- sample(x=1:V, size=n1, replace=TRUE)
17
18 for (v in 1:V) {
19   train.v <- data[id.fold!=v, ]; test.v <- data[id.fold==v, ];
20   mtry = tuneRF(train.v[, -1], factor(train.v$Category), ntreeTry=200,
21               stepFactor=2, improve=0.05, trace=TRUE,
22               plot=FALSE, dobest=FALSE, printFlag = FALSE)
23   best.mtry = mtry[mtry[, 2] == min(mtry[, 2]), 1]
24
25 ## Fitting model
26 fit.rf = randomForest(factor(Category) ~., mtry=best.mtry,
27                       data=train.v, importance=TRUE, proximity=TRUE,
28                       ntree=500)
29

```



```

30 yobs = test.v$Category
31 pred.rf = predict(fit.rf, newdata=test.v[, -c(1)], type="prob")[, 2]
32 area = roc.area(yobs, pred.rf)$A
33 error[v] = area
34 print(paste("AUC for fold", v, ":", error[v]))
35
36 pred.rate = ifelse(pred.rf > 0.5, 1, 0)
37 miss.rate <- mean(yobs != pred.rate)
38 missclass.rate[v] = miss.rate
39 print(paste("Missclassification rate for fold", v,
40             ":", missclass.rate[v]))
41 }
42
43 print(paste("Average of AUC:", mean(error)))
44 print(paste("Average of Miss:", mean(missclass.rate)))
45
46 rf.miss<-mean(missclass.rate)
47 rf.AUC<-mean(error)
48 rf.AUC
49
50 # FITTING THE RF MODEL
51 set.seed(125)
52 fit.rf
53
54 pred.rf = predict(fit.rf, newdata=test.v[, -1], type="prob")[, 2]
55
56 # VARIABLE IMPORTANCE RANKING
57 round(importance(fit.rf), 2)
58 varImpPlot(fit.rf, main="Variable Importance Ranking")
59
60 # AREA UNDER THE CURVE OF RANDOM FOREST MODEL
61 AUC.RF <- roc.area(obs=yobs, pred=pred.rf)$A
62 area.rf <- verify(obs=yobs, pred=pred.rf)
63 roc.plot(area.rf, plot.thres = NULL, col="red", main="ROC Curve from
64           Random Forest")
65 text(x=0.7, y=0.2, paste("Area under ROC =", round(AUC.RF, digits=6),
66                          sep=" "), col="blue", cex=0.9)

```

```

1 # MULTIVARIATE ADAPTIVE REGRESSION SPLINES THROUGH V-FOLD
2 set.seed(125)
3 library("earth")
4 library(ggplot2) # plotting
5 library(caret) # automating the tuning process
6 library(vip) # variable importance
7 library(pdp) # variable relationships
8
9 set.seed(125)
10 V <- 10
11 n <- NROW(data); n0 <- sum(data$Category==0); n1 <- n-n0;
12 missclass.rate = c()
13 error=c()
14 for (v in 1:V) {
15   error=c(error, v)
16   missclass.rate=c(missclass.rate, v)
17 }
18
19 id.fold <- 1:n
20 id.fold[data$Category==0] <- sample(x=1:V, size=n0, replace=TRUE)

```

```

21 id.fold[data$Category==1] <- sample(x=1:V, size=n1, replace=TRUE)
22
23 for (v in 1:V) {
24   train.v <- data[id.fold!=v, ]; test.v <- data[id.fold==v, ];
25   fit.mars <- earth(Category ~ ., data = train.v, degree=3,
26                     glm=list(family=binomial(link = "logit")))
27
28   yobs = test.v$Category
29   yhat.mars <- predict(fit.mars, newdata=test.v[, -1], type="response")
30   area = roc.area(yobs, yhat.mars)$A
31   error[v] = area
32   print(paste("AUC for fold", v, ":", error[v]))
33
34   pred.rate = ifelse(pred.rf > 0.5, 1, 0)
35   miss.rate <- mean(yobs != pred.rate)
36   missclass.rate[v] = miss.rate
37   print(paste("Missclassification rate for fold", v,
38               ":", missclass.rate[v]))
39 }
40
41 print(paste("Average of AUC:", mean(error)))
42 print(paste("Average of Miss:", mean(missclass.rate)))
43
44 MARS.miss<-mean(missclass.rate)
45 MARS.AUC<-mean(error)
46
47 # VARIABLE IMPORTANCE PLOT
48 vip(fit.mars, num_features = 10) + ggtitle("GCV")
49
50 # PREDICTION
51 AUC.MARS <- ci.cvAUC(predictions=yhat.mars, labels=yobs,
52                     folds=1:length(yhat.mars), confidence=0.95); AUC.
53                     MARS
54
55 AUC.Mar<-AUC.MARS$cvAUC
56 auc.ci <- round(AUC.MARS$ci, digits=6)
57 library(verification)
58 area.mars <- verify(obs=yobs, pred=yhat.mars)
59 roc.plot(area.mars, plot.thres = NULL, main="ROC Curve from MARS")
60 text(x=0.6, y=0.2, paste("Area under ROC =",
61                           round(AUC.MARS$cvAUC, digits=4),
62                           sep=" "), col="cadetblue", cex=0.9)
63
64 # Missclassification rate
65 missRate.Mars <- mean(yobs != (yhat.mars>0.5))
66 missRate.Mars

```

```

1 # ARTIFICIAL NEURAL NETWORKS
2 # (5d) Fit at least two different artificial neural network models
3
4 # DATA PREPARATION - NEED TO DEAL WITH CATEGORICAL PREDICTORS
5 X.dat <- as.data.frame(model.matrix(Category~.-1, data=data))
6 dat1 <- data.frame(cbind( Category=data$Category,X.dat))
7
8 # Model 1: 1 hidden layer, 3 units
9 set.seed(125)
10 library(neuralnet);
11 V <- 10

```

```

12 index.cv <- sample(1:V, size=NROW(dat1), replace=TRUE)
13 missclass.rate = c()
14 error=c()
15 for (v in 1:V) {
16   error=c(error, v)
17   missclass.rate=c(missclass.rate, v)
18 }
19
20 for (v in 1:V){
21   print(v)
22   train_d.v <- dat1[index.cv!=v, ]
23   valid_d.v <- dat1[index.cv==v, ]
24   X.train <- train_d.v[, -1]; X.test <- valid_d.v[, -1]
25   scale.train <- scale(X.train, center=TRUE, scale = TRUE)
26   train_scaled.v <- data.frame(Category=train_d.v[, 1] , scale.train)
27   valid_d.v[, 2:14] <- as.data.frame(scale(X.test,
28                                           center=attributes(scale.
29                                           train)$'scaled:center',
30                                           scale=attributes(scale.train
31                                           )$'scaled:scale'))
32
33   yobs <- valid_d.v[, 1]
34
35   # Using 1 hidden layer, 3 units
36   options(digits=3)
37   net1 = neuralnet(Category~., data=train_scaled.v, hidden=3, rep = 1,
38                     threshold = 0.05, stepmax = 1e+05, algorithm = "
39                     rprop+",
40                     err.fct = "ce", act.fct = "logistic",
41                     linear.output=FALSE, likelihood=TRUE)
42
43   pred.11 = as.vector(neuralnet::compute(net1,
44                                           covariate=valid_d.v[, -1])$net.
45                                           result)
46
47   area = roc.area(yobs, pred.11)$A
48   error[v] = area
49   print(paste("AUC for fold", v, ":", error[v]))
50
51   pred.rate = ifelse(pred.11 > 0.5, 1, 0)
52   miss.rate <- mean(yobs != pred.rate)
53   missclass.rate[v] = miss.rate
54   print(paste("Missclassification rate for fold", v,
55               ":", missclass.rate[v]))
56 }
57
58 print(paste("Average of AUC:", mean(error)))
59 print(paste("Average of Miss:", mean(missclass.rate)))
60
61 ANN1.miss<-mean(missclass.rate)
62 ANN1.AUC<-mean(error)
63
64 # Model 2: 2 hidden layer, 3 units
65 set.seed(125)
66 library(neuralnet);
67 V <- 10
68 index.cv <- sample(1:V, size=NROW(dat1), replace=TRUE)
69 missclass.rate = c()
70 error=c()
71 for (v in 1:V) {

```

```

66 error=c(error, v)
67 missclass.rate=c(missclass.rate, v)
68 }
69
70 for (v in 1:V){
71   print(v)
72   train_d.v <- dat1[index.cv!=v, ]
73   valid_d.v <- dat1[index.cv==v, ]
74   X.train <- train_d.v[, -1]; X.test <- valid_d.v[, -1]
75   scale.train <- scale(X.train, center=TRUE, scale = TRUE)
76   train_scaled.v <- data.frame(Category=train_d.v[, 1] , scale.train)
77   valid_d.v[, 2:14] <- as.data.frame(scale(X.test,
78                                           center=attributes(scale.
79                                           train)$'scaled:center',
80                                           scale=attributes(scale.train
81                                           )$'scaled:scale'))
82
83   yobs <- valid_d.v[, 1]
84
85   # Using 2 hidden layer, 3 units
86   options(digits=3)
87   net2 = neuralnet(Category~., data=train_scaled.v, hidden=c(2,3), rep
88   = 1,
89   threshold = 0.05, stepmax = 1e+05, algorithm = "
90   rprop+",
91   err.fct = "ce", act.fct = "logistic",
92   linear.output=FALSE, likelihood=TRUE)
93
94   pred.11 = as.vector(neuralnet::compute(net2,
95   covariate=valid_d.v[, -1])$net.
96   result)
97
98   area = roc.area(yobs, pred.11)$A
99   error[v] = area
100   print(paste("AUC for fold", v, ":", error[v]))
101
102   pred.rate = ifelse(pred.11 > 0.5, 1, 0)
103   miss.rate <- mean(yobs != pred.rate)
104   missclass.rate[v] = miss.rate
105   print(paste("Missclassification rate for fold", v,
106   ":", missclass.rate[v]))
107 }
108
109 print(paste("Average of AUC:", mean(error)))
110 print(paste("Average of Miss:", mean(missclass.rate)))
111
112 ANN2.miss<-mean(missclass.rate)
113 ANN2.AUC<-mean(error)
114
115 # Model 3: 5 hidden layer, 4 units
116 library(neuralnet);
117 set.seed(125)
118 V <- 10
119 index.cv <- sample(1:V, size=NROW(dat1), replace=TRUE)
120 missclass.rate = c()
121 error=c()
122 for (v in 1:V) {
123   error=c(error, v)
124   missclass.rate=c(missclass.rate, v)
125 }

```

```

119
120 for (v in 1:V){
121   print(v)
122   train_d.v <- dat1[index.cv!=v, ]
123   valid_d.v <- dat1[index.cv==v, ]
124   X.train <- train_d.v[, -1]; X.test <- valid_d.v[, -1]
125   scale.train <- scale(X.train, center=TRUE, scale = TRUE)
126   train_scaled.v <- data.frame(Category=train_d.v[, 1] , scale.train)
127   valid_d.v[, 2:14] <- as.data.frame(scale(X.test,
128                                           center=attributes(scale.
129                                           train)$'scaled:center',
130                                           scale=attributes(scale.train
131                                           )$'scaled:scale'))
132
133   yobs <- valid_d.v[, 1]
134
135   # Using 5 hidden layer, 4 units
136   options(digits=3)
137   net3 = neuralnet(Category~., data=train_scaled.v, hidden=c(5,4), rep
138     = 1,
139     threshold = 0.05, stepmax = 1e+05, algorithm = "
140       rprop+",
141       err.fct = "ce", act.fct = "logistic",
142       linear.output=FALSE, likelihood=TRUE)
143
144   pred.11 = as.vector(neuralnet::compute(net3,
145                                           covariate=valid_d.v[, -1])$net.
146                                           result)
147
148   area = roc.area(yobs, pred.11)$A
149   error[v] = area
150   print(paste("AUC for fold", v, ":", error[v]))
151
152   pred.rate = ifelse(pred.11 > 0.5, 1, 0)
153   miss.rate <- mean(yobs != pred.rate)
154   missclass.rate[v] = miss.rate
155   print(paste("Missclassification rate for fold", v,
156     ":", missclass.rate[v]))
157 }
158
159 print(paste("Average of AUC:", mean(error)))
160 print(paste("Average of Miss:", mean(missclass.rate)))
161
162 ANN3.miss<-mean(missclass.rate)
163 ANN3.AUC<-mean(error)

```

```

1 # SUPPORT VECTOR MACHINE (SVM)
2 # (5e) Fit at least two different SVM models
3
4 # Using Linear Kernel
5 set.seed(125)
6 library(caret)
7 data11 = as.data.frame(model.matrix(Category~. -1, data = data))
8 newdata = data.frame(Category = data$Category, data11)
9 set.seed(125)
10 V <- 10
11 index.cv <- sample(1:V, size=NROW(newdata), replace=TRUE)
12 missclass.rate = c()
13 error=c()
14 for (v in 1:V) {error=c(error, v)

```

```

15   missclass.rate=c(missclass.rate, v)
16 }
17
18 for (v in 1:V){
19   print(v)
20   train_d.v <- newdata[index.cv!=v, ]
21   valid_d.v <- newdata[index.cv==v, ]
22   yobs <- valid_d.v[, 1]
23   X.train <- train_d.v[, -1]; X.test <- valid_d.v[, -1]
24   scale.train <- scale(X.train, center=TRUE, scale = TRUE)
25   train_scaled.v <- data.frame(Category=train_d.v[, 1] , scale.train)
26   valid_d.v[, 2:14] <- as.data.frame(scale(X.test,
27                                           center=attributes(scale.
28                                           train)$'scaled:center',
29                                           scale=attributes(scale.train
30                                           )$'scaled:scale'))
31
32   trctrl <- trainControl(method = "repeatedcv", number=10, repeats = 3)
33   svm_Linear <- train(factor(Category) ~.,
34                       data =train_scaled.v , method = "svmLinear",
35                       trControl=trctrl,
36                       tuneLength = 10)
37
38   test_pred.svm1 <- predict(svm_Linear, newdata = valid_d.v[, -1])
39   test_pred.svm1 <- as.numeric(as.character(test_pred.svm1))
40   area = roc.area(yobs, test_pred.svm1)$A
41   error[v] = area
42   print(paste("AUC for fold", v, ":", error[v]))
43
44   pred.rate = ifelse(test_pred.svm1 > 0.5, 1, 0)
45   miss.rate <- mean(yobs != pred.rate)
46   missclass.rate[v] = miss.rate
47   print(paste("Missclassification rate for fold", v, ":",
48               missclass.rate[v]))
49 }
50
51 averageAUC = print(c("Average AUC:", mean(error)))
52 print(c("Average Misclassification rate:", mean(missclass.rate)))
53
54 SVM1.miss<-mean(missclass.rate)
55 SVM1.AUC<-mean(error)
56
57 # C VALUE IN LINEAR CLASSIFIER SVM
58 library(caret)
59 set.seed(125)
60 data11 = as.data.frame(model.matrix(Category~. -1, data = data))
61 newdata = data.frame(Category = data$Category, data11)
62 V <- 10
63 index.cv <- sample(1:V, size=NROW(newdata), replace=TRUE)
64 missclass.rate = c()
65 error=c()
66 for (v in 1:V) {error=c(error, v)
67   missclass.rate=c(missclass.rate, v)
68 }
69
70 for (v in 1:V){
71   print(v)
72   train_d.v <- newdata[index.cv!=v, ]

```

```

70 valid_d.v <- newdata[index.cv==v, ]
71 yobs <- valid_d.v[, 1]
72 X.train <- train_d.v[, -1]; X.test <- valid_d.v[, -1]
73 scale.train <- scale(X.train, center=TRUE, scale = TRUE)
74 train_scaled.v <- data.frame(Category=train_d.v[, 1] , scale.train)
75 valid_d.v[, 2:14] <- as.data.frame(scale(X.test,
76                                     center=attributes(scale.
77                                     train)$'scaled:center',
78                                     scale=attributes(scale.train
79                                     )$'scaled:scale'))
80
81 grid <- expand.grid(C = c(0,0.01, 0.05, 0.1, 0.25, 0.5, 0.75, 1,
82                          1.25, 1.5, 1.75, 2,5))
83 svm_Linear_Grid <- train(factor(Category) ~., data = train_scaled.v,
84                          method = "svmLinear",
85                          trControl=trctrl,
86                          tuneGrid = grid,
87                          tuneLength = 10)
88
89 test_pred.svm2 <- predict(svm_Linear_Grid, newdata =valid_d.v[,-1],
90                          type="raw")
91 test_pred.svm2 <- as.numeric(as.character(test_pred.svm2

```